ASTRA OS SCHEDULER: TERMINAL EXECUTION RUNBOOK

Step-by-Step Operator Guide for Live Verification & GitHub Deployment

POSIX C++20 Architecture | Real-Time Hardware Telemetry | Release Build

Pre-Flight Workspace Setup: Working Directory & Environment

Open your Ubuntu / WSL2 terminal and navigate directly to the project root directory before executing commands:

```
cd "/mnt/c/Users/ASHAY_GUPTA/Desktop/OS GPU Scheduler"
pwd && ls -lh build/astra
```

Verification: Confirms current working directory is `OS GPU Scheduler` and the binary `./build/astra` is present.

Stage 1: Hardware Telemetry, Build Verification & Automated CTest Suite

Step 1: Clean Compilation & Build Verification (If Rebuilding Needed)

To ensure the binary reflects all latest optimizations and zero-synthetic telemetry fixes, compile with Ninja:

```
cmake --build build -j$(nproc)
```

- **Expected Output:** `[14/14] Linking CXX executable astra` with exit code 0.

Step 2: Live Hardware Probing & Real System Diagnostics

Query host POSIX subsystem parameters and live GPU telemetry:

```
./build/astra version
```

- **Dynamic Probing Mechanism:** Reads online cores via `sysconf(_SC_NPROCESSORS_ONLN)`, memory page size via `sysconf(_SC_PAGESIZE)`, host RAM from `/proc/meminfo`, and invokes `nvidia-smi` via `popen()` to query physical VRAM.
- **Expected Output:** ASCII banner followed by C++20 compiler version, online CPU cores, 4096-byte page size, total host RAM, and your physical **NVIDIA GeForce RTX 3050 Laptop GPU (4096 MB VRAM)**.

Step 3: Automated CTest 10-Phase Quality Gate (Full Test Suite)

Execute all 10 unit and integration phases through CMake's native test harness:

```
ctest --test-dir build --output-on-failure
```

- **Phase Breakdown:**
  - `Phase0Sanity` (0.05s): Core data structure integrity and environment validation.
  - `Phase1DomainAndStorage` (0.33s): SQLite persistence, Job FSM states, transactional journal.
  - `Phase2CLI` (0.23s): Command-line parser, argument dispatch, error handling.
  - `Phase3WorkersAndIPC` (2.05s): Worker daemon pooling, `AF_UNIX` domain sockets, binary protocol.
  - `Phase4SchedulerCore` (3.81s): FCFS, Strict Priority, SJF, and Priority Aging preemptive logic.
  - `Phase5TUIAndTelemetry` (1.17s): ANSI terminal rendering, gauge calculations, resource snapshots.
  - `Phase6DAGWorkflows` (10.16s): Kahn's topological sort, diamond DAG parallel execution, cycle rejection.
  - `Phase7ReliabilityAndDeadlockLab` (0.97s): Banker's algorithm, WFG cycles, explainable preemption.
  - `Phase8GPUAndCheckpointing` (1.18s): Non-volatile checkpoints, atomic state serialization, crash resume.
  - `Phase9ReplayBenchmarksAndRelease` (0.43s): Synthetic trace generator, Gantt timeline reconstruction.
- **Expected Result:** **100% tests passed, 0 tests failed out of 10** in $\sim$ 20.7 seconds.

Stage 2: OS Deadlock Laboratory & Fault-Tolerant Checkpointing

Step 4: OS Deadlock Laboratory Simulation (Banker's Algorithm & WFG Visualizer)

Execute the full 7-section academic deadlock detection, circular wait analysis, and preemption lab:

```
./build/astra deadlock demo
```

- **Section [1] Process State & Contention Matrix:** Shows `Job-Alpha` holding CPU Core 0 and waiting for GPU, while `Job-Beta` holds GPU and waits for CPU Core 0 across worker thread slots.
- **Section [2] Shared Resource Inventory:** Evaluates contention load (200% over-subscribed) across CPU Core 0 and the detected **NVIDIA GeForce RTX 3050 Laptop GPU**.
- **Section [3] Deadlock Risk Analysis:** Mathematical contention risk meter calculated as:

$$\text{Risk}(J) = 1.0 - \frac{\text{Available}_r}{\text{Need}_r} = 1.0 - \frac{0}{1} = 100.0\% \quad \text{[CRITICAL CONTENTION / 0.0\% SAFETY MARGIN]}$$

- **Section [4] Wait-For Graph (WFG) Visualization:** Renders the circular ASCII hold-and-wait dependency. Triggers 3-color DFS recursion: `CIRCULAR WAIT DETECTED IN DFS: [Job-Beta] -> [Job-Alpha] -> [Job-Beta]`.
- **Section [5] Dijkstra's Banker's Safety Matrix:** Proves Available $= [0, 0]$, Need $= [0, 1]$ and $[1, 0]$ with zero safe sequences.
- **Section [6] Explainable Preemption & Victim Selection:** Evaluates cost-loss optimization function:

$$\text{CostScore} = (P_{\text{base}} \cdot 0.70) + (\tau_{\text{sunk}} \cdot 0.30)$$

  - **Job-Alpha (Pri 8, Sunk 1450ms):** $(8 \cdot 0.70) + (1450 \cdot 0.30) = 440.6$
  - **Job-Beta (Pri 2, Sunk 120ms):** $(2 \cdot 0.70) + (120 \cdot 0.30) = \mathbf{37.4} \implies$ **Optimal Minimal-Loss Victim**
- **Section [7] Post-Recovery Safety Verification:** Preempts Job-Beta, releases RTX 3050 GPU back to pool, restores safety score to 100.0% SAFE with provably safe execution sequence $\langle$Job-Alpha$\rangle$.

Step 5: Atomic Checkpoint & Crash Recovery (Fault Tolerance)

Verify non-volatile checkpointing and exact resume mechanics after a process fault:

```
# 1. Run compute workload configured to simulate fatal crash at step 50,000 (Target:
    100,000)
./build/workloads/checkpointable_sum --target 100000 --fail-at 50000 --checkpoint-file /
    tmp/test.ckpt

# 2. Inspect serialized binary state checkpoint file
ls -lh /tmp/test.ckpt

# 3. Resume computation from preserved state checkpoint
./build/workloads/checkpointable_sum --target 100000 --resume --checkpoint-file /tmp/test.
    ckpt
```

- **Operating System Principle:** Atomically records iteration counter and partial accumulator to disk before crash. On resume, starts directly at iteration 50,001 without recalculating the first 50,000 iterations.
- **Expected Output:** Command 1 terminates with `SIMULATED CRASH triggered at step 50000`. Command 3 prints `RESUMING from checkpoint at step 50000` and completes final sum $(5,000,050,000)$ in < 0.05s.

Stage 3: Policy Benchmarks, DAGs, TUI & GitHub Deployment

Step 6: Empirical Scheduling Policy Benchmark (Convoy Effect & Aging)

Simulate 20 synthetic tasks across 2 concurrency slots to evaluate FCFS, Strict Priority, Priority Aging, and SJF:

```
./build/astra bench --jobs 20 --slots 2
```

- **Theoretical OS Findings Displayed:** FCFS suffers Convoy Effect behind Job 0 (Avg Wait: 10.81s); Strict Priority starves low-priority jobs (Max Wait: 20.90s); Priority Aging dynamically elevates priority, achieving **0 starved tasks**; SJF minimizes average turnaround time to **11.17s**.

Step 7: DAG Workflow Parallel Execution & Cycle Rejection

Test Kahn's parallel dispatch and 3-color DFS cycle rejection:

```
# Parallel Diamond DAG: Stages B & C execute concurrently across worker slots
./build/astra workflow submit examples/diamond_workflow.json

# Cyclic Workflow Rejection: Validates 3-Color DFS cycle rejection (Exit code 1)
./build/astra workflow submit examples/cyclic_workflow.json
```

Step 8: Interactive Real-Time Terminal Dashboard (TUI) & Visualizers

Launch the live ncurses/POSIX terminal monitoring dashboard:

```
./build/astra top
```

- **View Switching & Controls:**
  - [**Tab**] / **Key Shortcuts:** Cycles across views: `[j]` Jobs Dashboard, `[g]` Gantt Timeline & TAT, `[h]` Host OS `/proc` Table, `[w]` Worker Pool, `[e]` Audit Events.
  - [**Up**] / [**Down**] **Arrows:** Navigate through process rows. Selected row is highlighted with a cyan indicator.
  - [**Enter**] **Per-Process Inspection:** Opens the dedicated **Process Lifecycle Gantt Inspector** for the highlighted process. Renders chronological state progression, interrupt detection (`[*** INTERRUPT: FAULT ***]`), automated checkpoint preservation, and auto-resume (`[=== AUTO-RUN AGAIN ===]`). Press [**Esc**] to return.
  - [**s**] **Demo Workloads:** Submits synthetic test jobs directly from within the TUI.
  - [**c**] **Cancel:** Sends SIGINT / cancellation interrupt to the selected running job.
  - [**q**] **Quit:** Safely shuts down the TUI and returns to your shell.

Step 9: Dedicated CLI Visualizers (Gantt, Host OS & Per-Process Inspection)

Execute non-interactive terminal visualizers directly from your shell:

```
# 1. System-wide Real-Time Gantt Chart & TAT Breakdown
./build/astra gantt

# 2. Host Linux Kernel Process Table (/proc Inspector)
./build/astra host

# 3. Deep Process Lifecycle & Gantt Inspection (Pass Host PID or Astra Job ID)
./build/astra inspect 1
./build/astra inspect <job_id>
```

Step 10: Complete End-to-End Orchestrated Demonstration Script

Execute the unified demo script that runs through all subsystems sequentially:

```
bash scripts/demo.sh
```

Step 11: Clean Repository State & Safe Push to GitHub

Commit your clean codebase and documentation to your GitHub repository:

```
# 1. Clean up temporary test sockets and checkpoints
rm -f /tmp/astra_*.sock /tmp/astra_chk.bin /tmp/*.ckpt

# 2. Stage all source files, documentation, and Excalidraw architecture designs
git add . && git status

# 3. Commit with clean semantic message
git commit -m "feat(scheduler): per-process gantt lifecycle, host proc inspector, and
    updated architecture diagrams"

# 4. Push to remote main branch
git push origin main
```

Goal / Action	Exact Terminal Command	Expected Pass Criteria
Hardware Probe	`./build/astra version`	Detects CPU cores, RAM, and RTX 3050 GPU
Deadlock Lab	`./build/astra deadlock demo`	Solves circular wait via Banker's algorithm
Test Suite	`ctest -test-dir build`	10/10 tests pass (100% green in 18.7s)
Policy Benchmark	`./build/astra bench -jobs 20 -slots 2`	Evaluates FCFS, Priority, SJF, Aging
Gantt Chart	`./build/astra gantt`	Renders ASCII execution tracks & TAT formulae
Host OS Table	`./build/astra host`	Scans `/proc` kernel table (PIDs, CPU, RSS, state)
Process Inspect	`./build/astra inspect <id|pid>`	Renders per-process lifecycle timeline & Gantt bar
Diamond DAG	`./build/astra workflow submit examples/diamond_workflow.json`	Executes B & C in parallel; finishes D
Cycle Detection	`./build/astra workflow submit examples/cyclic_workflow.json`	Rejects cycle with 3-Color DFS error
Crash & Resume	`./build/workloads/checkpointable_sum -resume ...`	Resumes exactly from checkpoint
Interactive TUI	`./build/astra top`	Multi-view dashboard with `[Enter]` process inspect
Full Demo Script	`bash scripts/demo.sh`	Runs 7-step guided verification run

Table 1: **ASTRA OS Scheduler: Quick Command Reference Matrix.**

Operator Troubleshooting & Tips

- **Stale Unix Sockets:** If worker daemon fails with `Address already in use`, run: `rm -f /tmp/astra_*.sock`.
- **Clean Full Rebuild:** If you modify header files: `rm -rf build && cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release && cmake -build build`.